|array|string|float|int|bool|null. I'm hoping at least a few of you might have spotted an anomaly in that union declaration. resource has never been, and never will be, a standalone type.

Resources (which are variables containing references to operating system resources like file handles and network sockets) have always been a bit of a special case; they are also now phased out in favor of objects. Many resource-returning functions (from PHP 8.0 onwards) now return instances of resource classes, and the normal class type declarations should be used with these. For example, in PHP 8 curl_init() now returns an instance of the built-in class CurlHandle.

As you should expect, declaring a type of mixed means that *anything* (apart from void or none) is legal, even an old-style resource. In fact, it is the only way to get resources working with the PHP 8 type declaration system.

Return-Only Types

Finally, we come to the set of type declarations, which are only used for restricting what is returned from a function. These are as follows.

- void: the function returns control to the caller but no data.
- never: the function never returns control.

We'll look at the first two and save the final three for when we cover OOP. Firstly, void should be obvious. Use this when no data is returned (i.e., the final statement in the function block is return; – note the terminating semicolon). never should only be used for a function that never returns at all. Remember, a function without a return statement will still return control to the caller. The use case for never is a function that will terminate the script (e.g., with a die() or exit() statement). Here's an example of both in action:

```
// Only to be run in a CLI environment!
function shallWeContinue(): void
{
    $in = readline('Type something to continue, or press Enter on its own
    to abort' . PHP_EOL);
```